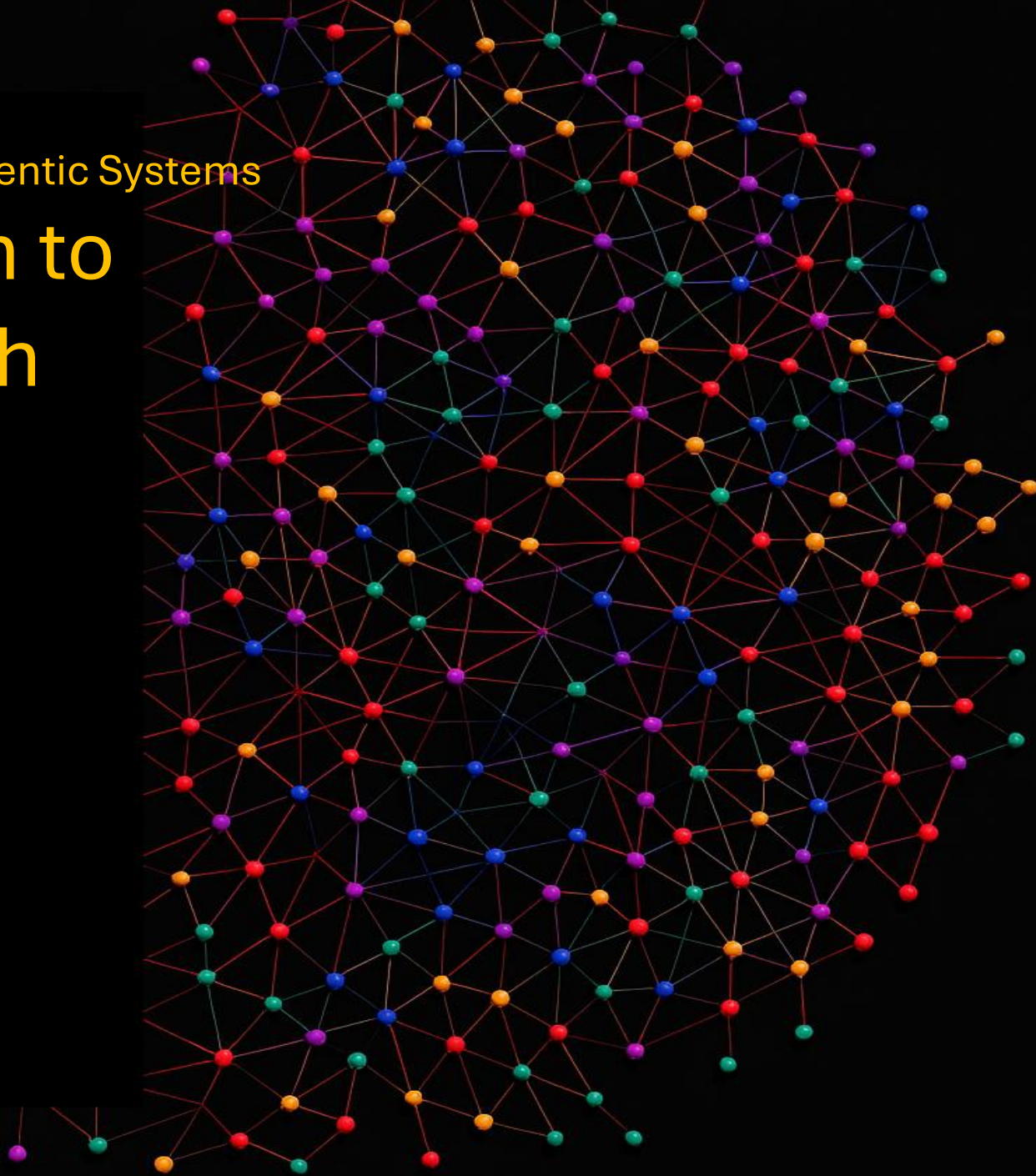


Programming Framework for Agentic Systems

Introduction to LangGraph

Siddhanna Janai
Ushadevi Alagappan





Agenda DAY1

- **What are Agents ?**
- **What is Agentic AI ?**
- **Key Components of Agentic AI**
- **Need for LangGraph**
- **What is LangGraph?**
- **LangGraph Components**
- **HON LAB – Notebooks Day 1**

What are Agents?

- Autonomous software entities designed to perform specific tasks based on goals, inputs, and contextual understanding.
- Can interpret user instructions, make decisions, and execute tasks independently or in collaboration with other agents and human supervisor.

What is Agentic AI?

Advanced systems that enable the creation and use of autonomous agents capable of planning, reasoning, adapting and collaborating.

Key Components of Agentic AI

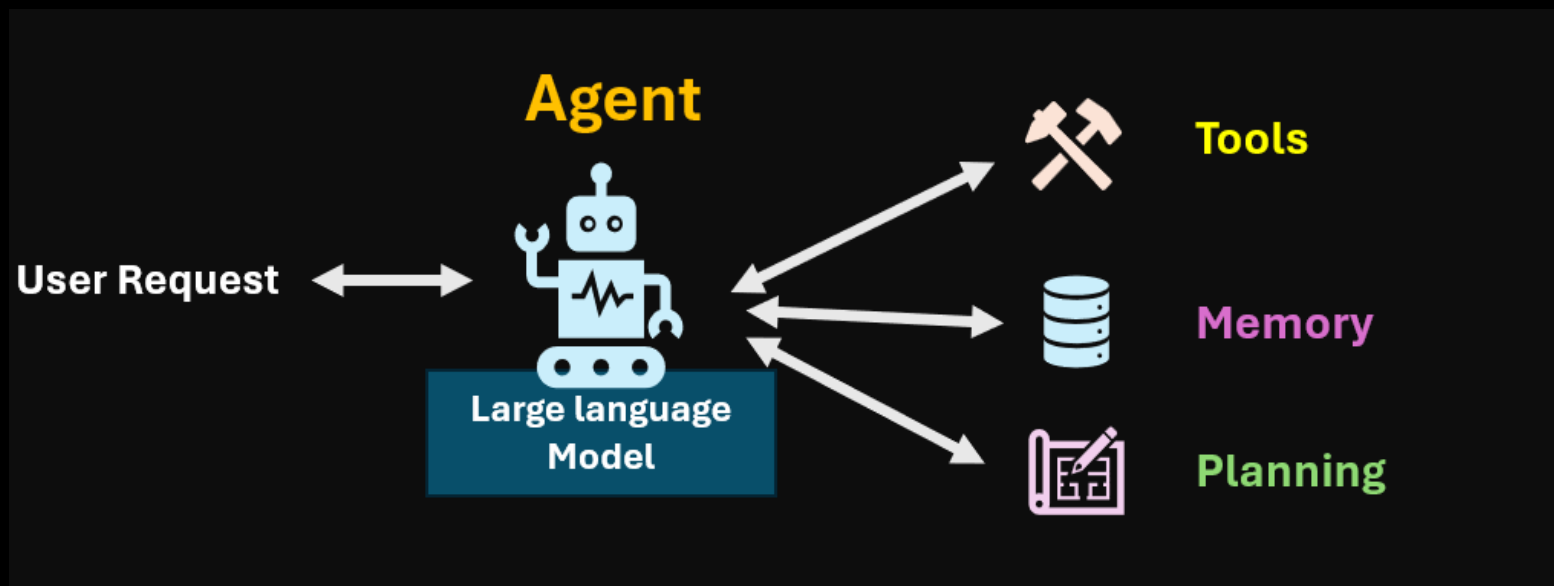
1. User Request

2. Agent

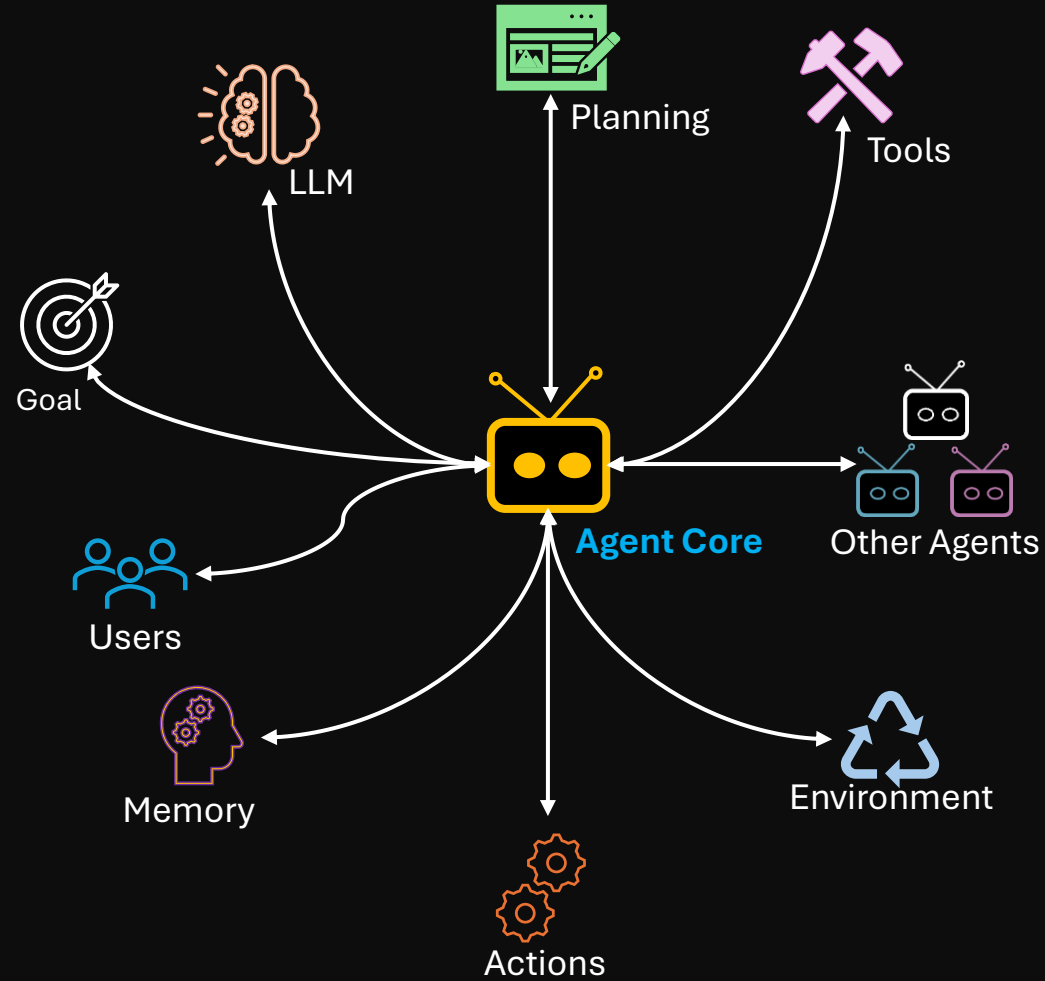
3.Memory

4.Tools

5.Planning



Agentic AI Architecture



Agent being a key component in Agentic AI, **what is it's role?**

The Role of Agents in Agentic AI

- **Task Orchestration:** Breaking down user requests into structured workflows.
- **Real-Time Adaptation:** Responding to changes in data or environment dynamically.
- **Multi-Agent Collaboration:** Solving problems by leveraging specialized skills of different agents.
- **Enhanced Automation:** Completing complex tasks with minimal human intervention by integrating with external tools and systems.



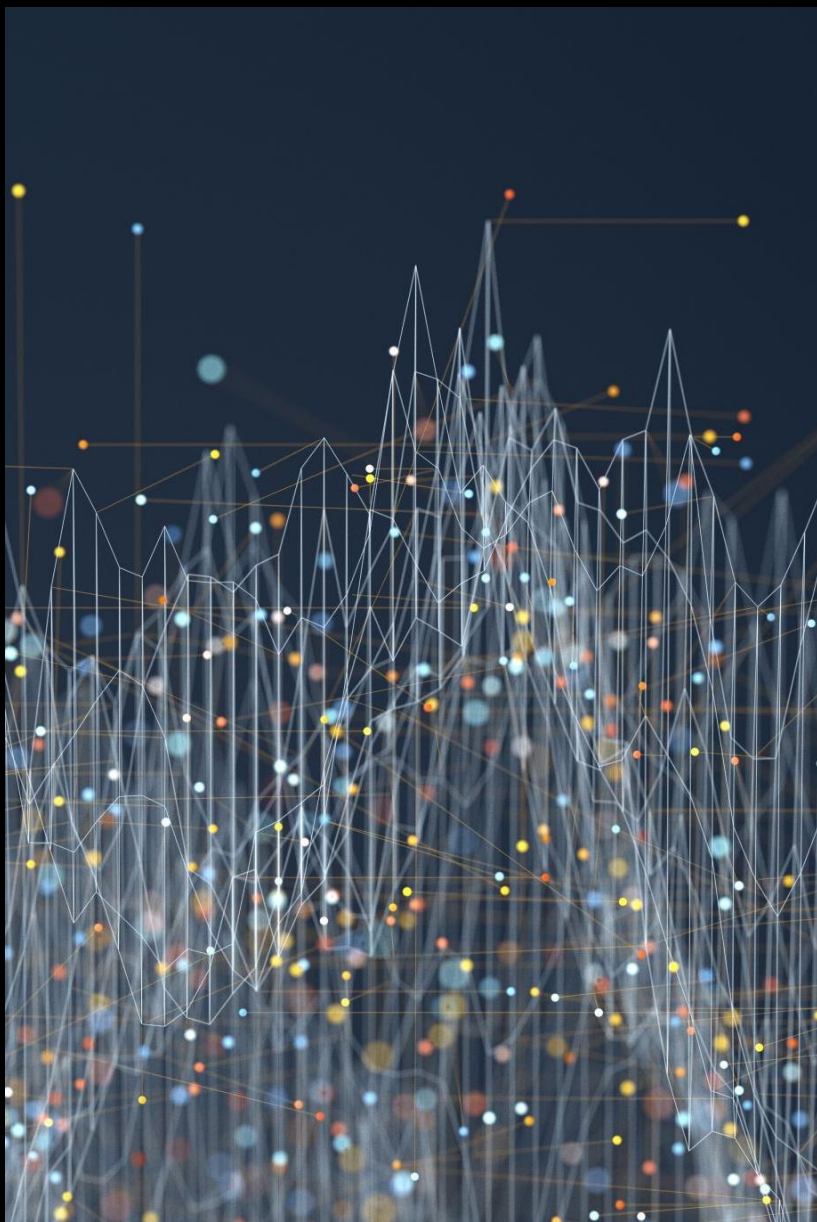
Need for LangGraph

Modern AI applications, especially those using LLMs like GPT or Claude, involve:

- Multi-turn conversations
- Tool integrations (e.g., calling APIs, summarizing docs)
- Decision-making based on context
- Dynamic branching and loops

While LLM is powerful for building the applications but a standalone LLM, is **inherently limited** for the following reasons:

- It lacks access to **external tools** (e.g., calculators, APIs).
- It doesn't have **contextual memory** or persistent state.
- It can't retrieve or reason over **external documents**.
- It struggles with **multi-step workflows** or **long-term planning**.
- In short, the model alone is **not autonomous** without orchestration.



Features of LangGraph

Graph-Based Architecture

LangGraph uses a **graph-based** design enabling **dynamic, adaptable agent workflows** beyond linear models.

Stateful and Branching Logic

It supports **stateful, multi-step, and branching logic** essential for sophisticated AI applications.

Modularity and Flexibility

LangGraph's modularity eases management of **complex interactions** and agent state transitions.

Enhanced Agentic AI

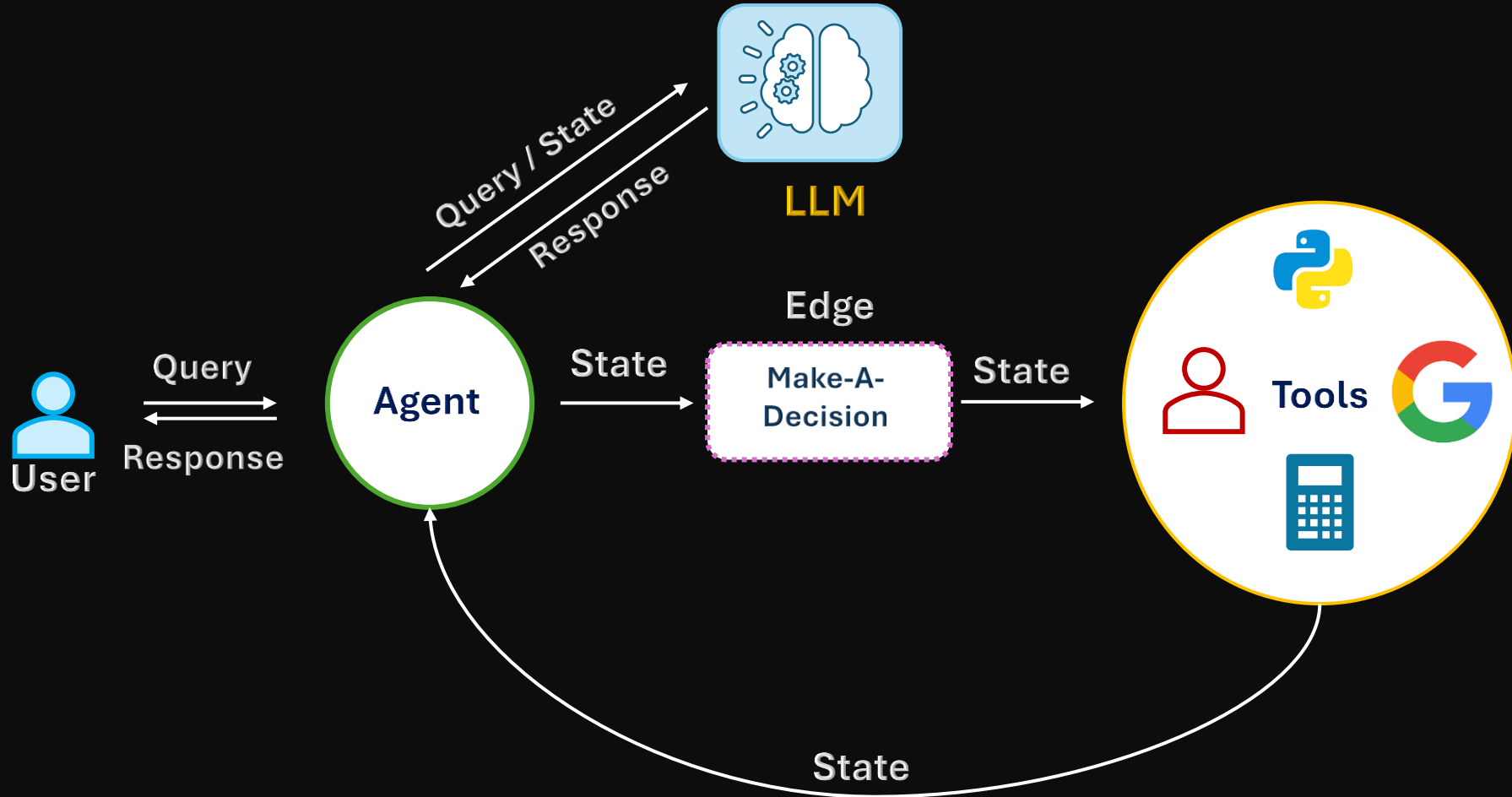
Extending LangChain, it **creates context-aware agents** capable of **autonomous, nuanced decision-making**.

LangGraph helps

- Orchestrate complex workflows.
- Manage state across multiple steps or agents.
- Build modular, scalable, and traceable pipelines.

What is LangGraph ?

LangGraph is a stateful agentic orchestration framework that enhances control over **agent workflows**.

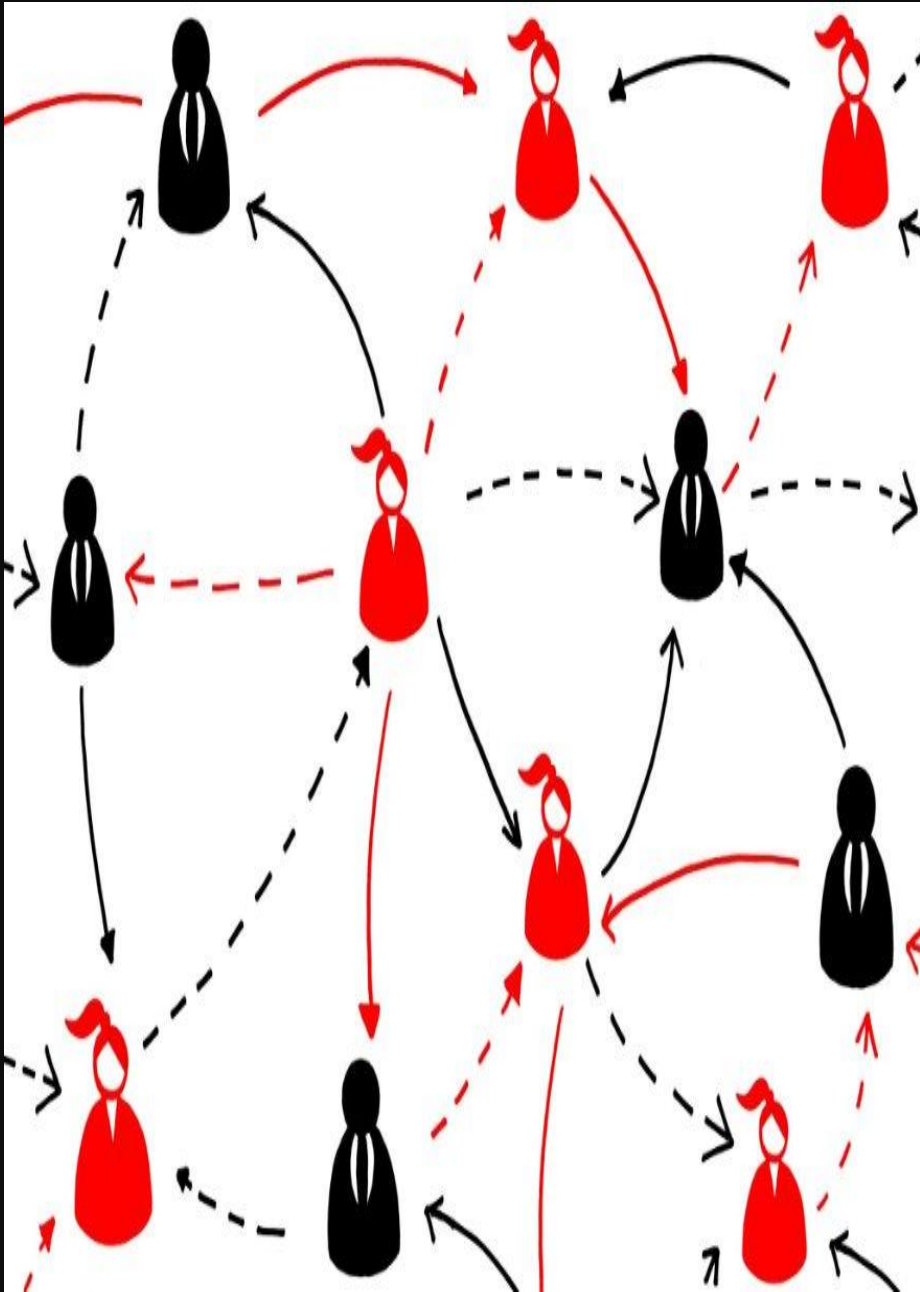


Agentic AI Frameworks

Feature	LangGraph	CrewAI	AutoGen	Agno
Architecture Style	Graph-based, stateful workflows	Role-based multi-agent collaboration	Feedback-driven agent loops	Modular, declarative agent design
Best Use Case	Complex decision trees, dynamic workflows	Team-based task execution	Adaptive reasoning with human-in-the-loop	Rapid prototyping of agent workflows
Memory & State Management	Built-in state transitions and memory	Limited memory, mostly stateless	Supports memory via feedback loops	Lightweight memory with declarative configs
Tool Integration	Seamless LangChain tools, custom APIs	Manual tool setup	Python, web search, APIs	Easy integration with external APIs
Control & Flexibility	Fine-grained control over agent flow	Moderate control via roles	High flexibility with feedback and loops	High flexibility with YAML-based configs

LangGraph stands out when you need **precise orchestration** and **dynamic agent behavior** in complex applications.

Aspect	LangGraph	Other Frameworks (e.g., CrewAI, AutoGen, Agno)
Workflow Design	Graph-based, enabling stateful, branching, and cyclic agent workflows	Mostly linear or role-based; limited support for complex branching or looping
Control & Customization	Offers fine-grained control over agent transitions, tools, and logic	Control varies; often more abstracted or declarative, less suited for deep logic



Use Cases

Multi-Agent Systems

LangGraph enables [collaboration among multiple AI agents](#) to achieve complex goals with seamless communication.

Conversational Agents with Memory

LangGraph supports [stateful conversations](#) by retaining and recalling context for natural, coherent interactions.

Decision Trees with LLMs

LangGraph builds [adaptive decision trees](#) powered by large language models for sophisticated decision-making.

Components of LangGraph

Dictionary

- Retrieves data efficiently using unique keys
- Flexible and easy to use
- No built-in checks for data type or structure

```
student = {"name": "Raj", "age": 21}  
print(student["name"]) # Raj
```

Typed Dictionary

- Defines fixed keys with specific data types
- Ensures data follows a set structure
- Helps catch type errors during development

```
from typing import TypedDict

class Student(TypedDict):
    name: str
    age: int

student = Student(name="Raj", age=21)
```

Union

- Allows a variable to hold values of multiple types
- Useful when data can be in more than one format

```
from typing import Union
```

```
StudentID = Union[int, str]
```

```
id1: StudentID = 101
```

```
id2: StudentID = "S101"
```

State

- Shared data structure holding the app's current context
- Like the app's memory, tracking variables and data

```
from typing import TypedDict

class State(TypedDict):
    query: str
    result: str

state = State(query="Hello", result="")
print(state["query"]) # Hello
```

StateGraph

- Core LangGraph component modeling the **workflow as a directed graph of nodes and edges**.
- Defines **how data (state) flows** between nodes, with optional conditional logic.

```
graph = StateGraph(State)
```

Node

- A single **processing unit / functional unit** in a workflow
- Takes input, performs a task, and outputs data

```
def greet_node(state):  
    state["greeting"] = f"Hello, {state['name']}!"  
    return state
```


Edge

- Connects nodes to define the flow of data or control
- Determines execution order in a workflow

```
graph.add_edge("tool_node", "prompt_node")
```

- Connects **tool_node** to **prompt_node**
- Ensures **prompt_node** runs after **tool_node** and receives its output

Conditional Edge

- Connects nodes while enabling dynamic decision-making in the workflow
- Determines the next node based on evaluated conditions

```
graph.add_conditional_edges("agent", should_continue, {  
    "tool": "tool",  
    "end": END,  
})
```

- From **agent**, routes to **tool** or **END** based on *should_continue function's o/p.*
- The mapping defines which branch each condition result triggers

Annotate & add_messages

- **Annotated** adds extra metadata to type hints in LangGraph
- **add_messages**: A utility to append messages dynamically to the state
- Ensures interactions (user queries, responses) are stored in the messages field

```
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages]
```

Messages



Human Message

Represents input from a user.



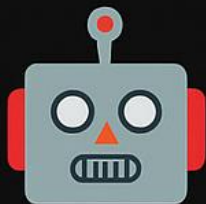
System Message

Used to provide instructions or context to the model



Function Message

Represents the result of a function call



AI Message

Represents responses



Tool Message

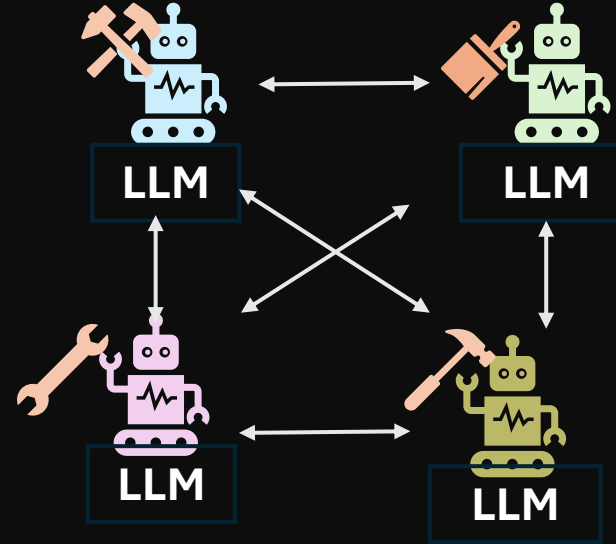
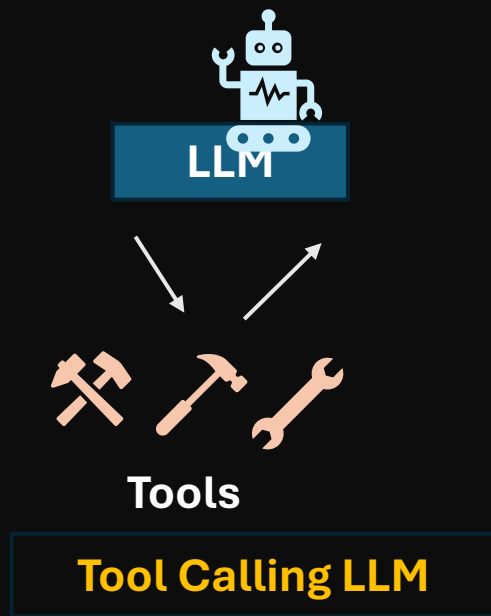
Similar to Function Message,



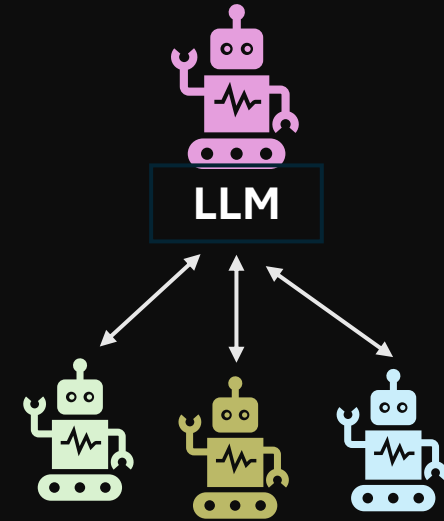
Agenda DAY 2

- **Agents With Tool Calling**
- **Multiple tool calls**
- **HON– Day2**

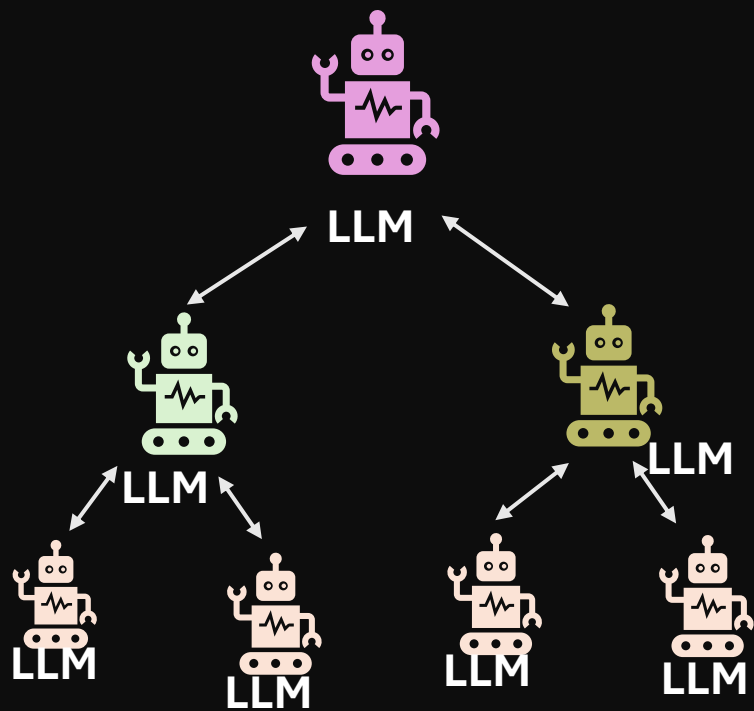
Multi-agent Agentic AI Networks



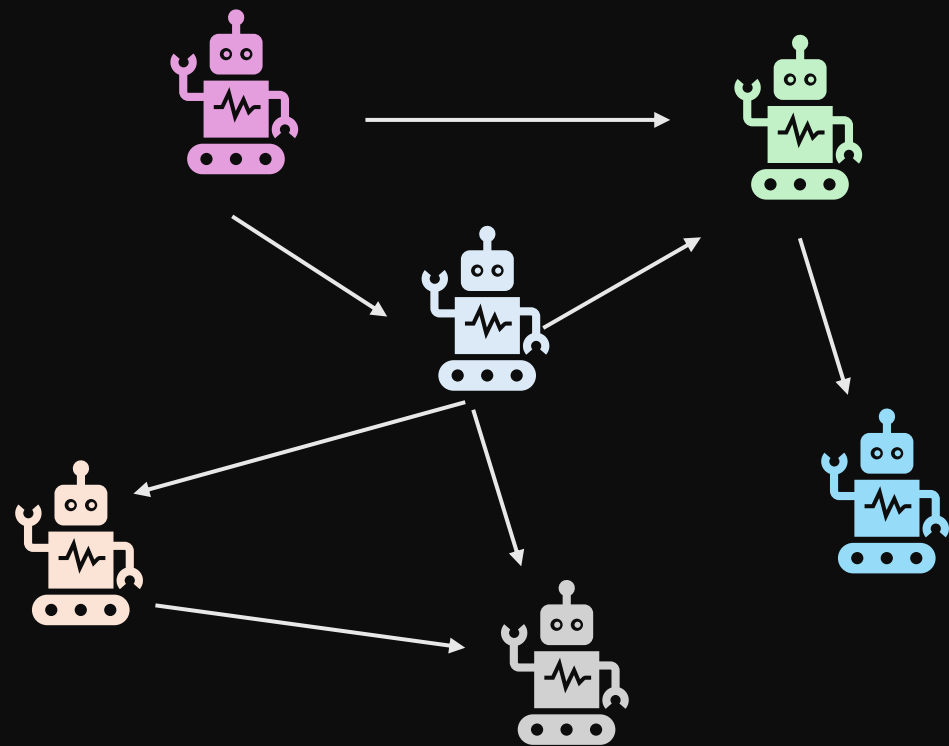
Network Architecture



Supervisor Architecture



Hierarchical Architecture



Custom Architecture

DEMO

LangGraph Agents